
Batch Normalization using an SVD Layer

Week 4: Consolidation and a Multi-Seed Replication of the Generative Result

Abdalmohsen Mohammedsaleh, Majed Alsulami, Yousef Alogiley,
Jad Mounayer (Mentor)

abdulmohsen1284@gmail.com, majedsaadalsulami@gmail.com, yousef.alogiley@gmail.com,
Jad.mounayer@outlook.com

Abstract

This report consolidates the first three weeks of our study of the Rank Reduction (RR) layer, an SVD based alternative to batch normalization, and subjects the single positive result of the project to a stronger test. Weeks 1 and 2 compared a plain convolutional baseline (OA), a batch normalized variant (OA_BN) and an RR variant (OA_RR) on a Gaussian blob localization task, where the plain baseline won and batch normalization was the weakest of the three. Week 3 moved to the autoencoder setting demonstrated in the reference repository and found the opposite ordering on latent space quality: the Rank Reduction Autoencoder (RRAE) produced far better random latent samples than the baseline, reported there as a factor of 7.9 from three seeds at a reduced training budget. In Week 4 we reimplemented that experiment from scratch and reran it over 8 seeds at 40 epochs. The direction of the Week 3 result replicates: RRAE wins random generation on 8 of 8 seeds, with a mean advantage of 18.0 times and a per seed range of 2.2 to 28.7 times. The width of that range is the main finding, since it shows the single number quoted in Week 3 was more precise than the evidence allows. The spread is structured rather than random: 5 seeds cluster near 27 times and the remaining 3 near 3 times, and the weak ones are exactly the seeds on which the RRAE itself converged poorly. Reconstruction remains the cost of the method, at a median factor of 21 times the error of the baseline, and batch normalization again fails to help on any metric. The takeaway is that the RR layer buys a well structured latent space, not accuracy, and that this benefit should be quoted as a range rather than a point estimate.

📅 **Date:** August 15, 2026

🔗 **Code:** <https://github.com/Yousef10p/RR-Layer/tree/new-experiment>

🌐 **Project Page:** https://github.com/JadM133/RR_layer

Contents

1	Introduction	2
2	Summary of Reports 1 to 3	2
2.1	Report 1: first comparison on the localization task	2
2.2	Report 2: reproducibility and ablations	3
2.3	Report 3: the autoencoder setting and the first win	3
3	Methodology	4
3.1	Data	4
3.2	Models	4
3.3	Metrics	4
3.4	Protocol	5
4	Results	5
4.1	Training behaviour	5
4.2	Reconstruction	6
4.3	Latent space quality	6
4.4	How large is the generative advantage, really	6
4.5	Two regimes across seeds	7
4.6	Why the RR layer generates better	8
5	Discussion	8
6	Code and Repository Status	9
7	Limitations and Future Work	10
8	Conclusion	10
	References	11

1 Introduction

Batch normalization is the standard tool for stabilizing the distribution of activations inside a deep network [1]. It works coordinate by coordinate: each feature is recentred and rescaled using statistics collected over the batch. The Rank Reduction (RR) layer studied in this project takes a different view of the same problem. Instead of normalizing each coordinate independently, it treats the batch as a matrix and replaces it with its best rank k approximation, computed by a truncated singular value decomposition and optimal in the Frobenius norm by the Eckart and Young theorem [2]. The activations are therefore constrained to a low dimensional subspace that is re-estimated from the data at every step, rather than being rescaled along fixed axes. This is the mechanism behind Rank Reduction Autoencoders [3], and the reference implementation we build on is our mentor's repository [4].

Problem statement. The question the project asks is narrow and empirical: on a controlled synthetic problem, does inserting an RR layer where one would normally insert a batch normalization layer help, and if so, on what. Three weekly reports have answered parts of this. Week 4 has two goals. The first is to bring the three answers together in one place, since the conclusions changed direction between Week 2 and Week 3 and a reader of any single report would come away with the wrong impression. The second is to stress test the one result that favours the RR layer, because a final report will lead with it and it currently rests on three seeds at a shortened training budget.

What is new in this report. Section 2 summarizes Reports 1 to 3. Sections 3 and 4 present a fresh implementation of the Week 3 autoencoder experiment, written from scratch rather than reused, and run over 8 seeds instead of three. An independent reimplementaion is a stronger check than rerunning the same code with more seeds, since it also exposes mistakes that are baked into the original script. Section 6 records the state of the two repositories and what still needs to be pushed.

Assumptions and scope. All results in this project come from one synthetic dataset of Gaussian blobs on a 64×64 grid, with a single latent width and a single RR rank. Nothing here should be read as a claim about the RR layer on natural images or at scale. The value of the setting is that the ground truth is known analytically, which is what makes the latent space metrics of Section 3 possible at all.

2 Summary of Reports 1 to 3

This section is a condensed record of the project so far. Nothing in it is new work; it exists so that the final report has one place to draw from, and so that the change of direction between Week 2 and Week 3 is visible at a glance.

2.1 Report 1: first comparison on the localization task

The first experiment placed a Gaussian blob of fixed width at a random centre inside a 64×64 image and trained a small convolutional network to regress the coordinates of that centre. Three variants shared an identical feature extractor and differed only in what sat between the extractor and the classifier head: nothing (OA), a batch normalization layer (OA_BN), or an RR layer of rank 8 (OA_RR). After 20 epochs the plain baseline was the most accurate by a wide margin, with a mean test displacement of 0.0026 against 0.0133 for the RR variant and 0.0335 for the batch normalized variant. The result was a single run with no seed averaging, so it established an

ordering but not its reliability.

2.2 Report 2: reproducibility and ablations

Report 2 answered mentor feedback on Report 1. The data generator was seeded so that runs were reproducible, every configuration was repeated over five seeds and reported as mean and standard deviation, batch normalization was reimplemented from scratch rather than taken from the framework, and the custom SVD backward pass was replaced by a thin wrapper around the native `torch.linalg.svd`. Architecture depth and training set size were both ablated. The ordering of Report 1 survived all of it: the plain baseline was best, the RR layer second, and batch normalization last, in every condition tested. At that point the honest summary of the project was that the RR layer was less harmful than batch normalization on this task, which is not the same as being useful.

2.3 Report 3: the autoencoder setting and the first win

Report 3 changed the task rather than the models. The reference repository demonstrates the RR layer inside an autoencoder, not a regressor, and the claim in the underlying paper is about the structure of the latent space rather than about accuracy [3]. We therefore rebuilt the experiment as an autoencoder with a 200 dimensional latent and an RR layer of rank 2, and evaluated three things instead of one: reconstruction of held out images, decoding of interpolated latent codes, and decoding of randomly sampled latent codes. The last of these is where the RR layer finally won, by a reported factor of 7.9 over the baseline on every seed and under both data protocols. Reconstruction was much worse for the RRAE at the 40 epoch budget used for the sweep, although a 200 epoch check showed the gap shrinking by roughly a factor of 8. Batch normalization improved neither metric under either protocol.

Report 3 also produced two side results worth carrying forward. The basis bank used by the reference implementation at inference time was validated: the basis assembled from the bank agreed with a basis computed from the full dataset to within about 1.4 degrees of principal angle, which means the extra `finalize_basis_from_data` routine added in our fork is unnecessary. That is a negative result, and it is reported as one. Three smaller issues in the reference code were also collected for the mentor: debug prints left live inside the SVD backward pass, the redundant routine just mentioned, and a spurious rank warning raised when a single sample is passed in evaluation mode.

Table 1 The project in three lines. The ordering reverses between the localization task and the generative evaluation.

Report	Task	Statistics	Ordering
1	Blob localization	1 run, 20 epochs	OA > OA_RR > OA_BN
2	Blob localization	5 seeds, ablations	OA > OA_RR > OA_BN
3	Autoencoding, generation	3 seeds, 40 epochs	RRAE > AE > AE_BN

Takeaway

Across every report so far the RR layer beats batch normalization. It beats the plain baseline only on the generative metric of Report 3, and that is the result Week 4 sets out to verify.

3 Methodology

3.1 Data

Each sample is a 64×64 image containing a single isotropic Gaussian of fixed width $\sigma = 0.2$, centred at a point drawn uniformly from $[-0.5, 0.5]^2$ on a domain that spans $[-1, 1]^2$. We draw 800 training images and 200 test images per seed, and standardize both with the mean and standard deviation of the training set. Centres are drawn continuously rather than placed on a grid, which is the protocol Report 3 identified as the fairer of the two. Every draw is seeded through `numpy.random.default_rng`, so a seed fixes the data and the initialization together.

The point of this dataset is that the ground truth is analytic. For any centre, including one that never appeared in training, we can construct the image the model should have produced. That is what makes the two latent space metrics below well defined.

3.2 Models

All three models share the same encoder, the same decoder and the same parameter count. The encoder is three convolution blocks with max pooling, $1 \rightarrow 8 \rightarrow 16 \rightarrow 32$ channels, followed by a linear map to a latent of width 200. The decoder mirrors it with a linear map and three transposed convolutions. The variants differ only in the layer applied to the latent vector:

- **AE:** identity, that is, no bottleneck layer at all.
- **AE_BN:** BatchNorm1d over the 200 latent units.
- **RRAE:** an RR layer of rank 2. During training it takes the SVD of the latent matrix of the batch, keeps the two leading components and reconstructs; during evaluation it projects onto a fixed basis assembled from the bank of the last twenty training bases.

The RR layer follows the reference implementation [4] with the native SVD wrapper adopted in Week 2. Training uses Adam [5] at 10^{-3} , batch size 32, mean squared reconstruction loss, for 40 epochs, in PyTorch [6] on CPU.

3.3 Metrics

Reconstruction error is the mean squared error between a held out test image and its reconstruction, in standardized data units.

The other two metrics ask whether the latent space is usable rather than whether the model memorizes its input, and both compare shapes rather than intensities: each image is standardized to zero mean and unit variance before the comparison, so a model is not rewarded or punished for the overall brightness of its output.

- **Interpolation error.** Take two test images, encode both, interpolate the two latent codes at $\alpha \in \{0.25, 0.5, 0.75\}$, and decode. The target is the analytic blob centred at the correspondingly interpolated centre. A latent space in which the two coordinates of the centre are encoded smoothly will decode interpolated codes into clean blobs at intermediate positions; a tangled one will not. We average over 100 random pairs.
- **Random generation error.** Fit a Gaussian to the latent codes of the training set, using the full covariance through an eigenvalue decomposition so that directions the model never uses receive no probability mass, draw 200 samples, and decode them. The target for each decoded image is the analytic blob at its own estimated centre, so the metric measures only whether the output looks like a valid blob at all, not where the blob landed. This is the quantitative version of the visual comparison in the reference notebook.

Both metrics were reimplemented for this report rather than copied from Week 3. That is deliberate, since an independent implementation is a stronger check, but it also means the absolute values here are not numerically comparable with the ones printed in Report 3. What is comparable is the ordering of the models and the ratios between them.

We also record the fraction of latent variance carried by the two leading principal directions, which explains the generation results rather than scoring them.

3.4 Protocol

Every model is trained on 8 seeds, 0 to 7, giving 24 runs in total. Results are reported as mean and standard deviation across seeds, and where a ratio between two models is quoted we also give its per seed range, since Report 3 quoted a mean ratio alone. Each finished run is written to `results.json` immediately, so the sweep is resumable.

4 Results

All numbers below come from the sweep described in Section 3: 8 seeds, 40 epochs, three models, 24 runs. Lower is better everywhere.

Table 2 Test metrics over 8 seeds, mean $\pm$ standard deviation. Reconstruction is in standardized data units; the interpolation and generation errors are shape errors as defined in Section 3.

Model	Reconstruction	Interpolation	Generation	Top 2 latent energy
AE (baseline)	0.0011 ± 0.0006	0.224 ± 0.055	0.152 ± 0.027	0.520 ± 0.038
AE_BN	0.0030 ± 0.0012	0.370 ± 0.046	0.417 ± 0.039	0.239 ± 0.008
RRAE ($k = 2$)	0.031 ± 0.025	0.145 ± 0.126	0.022 ± 0.024	1.000 ± 0.000

4.1 Training behaviour

Figure 1 shows the training loss of the three models averaged over seeds. The baseline and the batch normalized model converge to a similar training loss, while the RRAE plateaus about 23 times higher, which is the rank 2 constraint acting as a floor on how well the batch can be reconstructed rather than a failure to optimize.



Figure 1 Mean training loss per epoch over 8 seeds, log scale. The shaded band is one standard deviation across seeds.

4.2 Reconstruction

The baseline reconstructs best, at 0.00112. Batch normalization gives 0.00305, and the RRAE 0.0307. Expressed per seed, the RRAE costs a median factor of 21 against the baseline, with individual seeds from 5 to 117. Part of this is the expected price of forcing every batch through a two dimensional subspace, and Report 3 found that a large part of it closes at a longer training budget; the rest is the spread discussed in Section 4.5.

4.3 Latent space quality

This is where the models separate. On random generation the RRAE reaches 0.022 against 0.152 for the baseline, and it is better on every seed. On interpolation it reaches 0.145 against 0.224, better on 5 of 8 seeds. Batch normalization is the worst of the three on both, at 0.37 and 0.417, which means that on this problem adding it is worse than adding nothing.

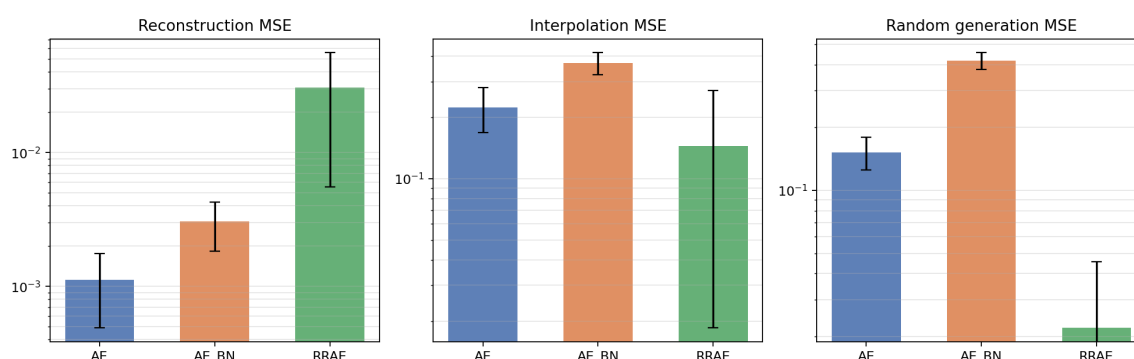


Figure 2 The three metrics side by side, mean over 8 seeds with standard deviation bars, log scale. The RR layer pays on the left panel and is paid back on the right two.

4.4 How large is the generative advantage, really

Report 3 quoted a single number, 7.9 times better than the baseline on random generation, from three seeds. Figure 3 shows the same comparison seed by seed at 8 seeds.

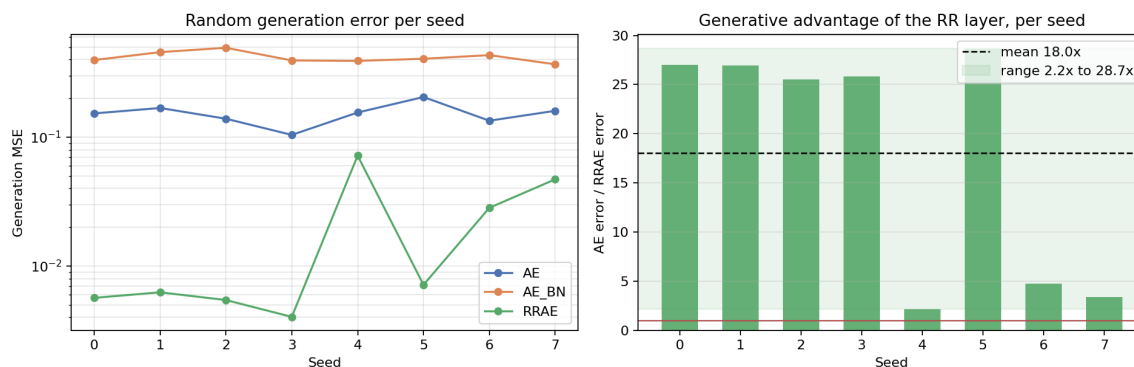


Figure 3 Left: random generation error per seed for the three models. Right: the ratio of baseline error to RRAE error, per seed. The dashed line marks the mean and the shaded band the full range.

The RRAE wins on 8 of 8 seeds, so the sign of the effect is not in doubt. Its size is: the mean ratio is 18.0, the median is 25.7, and the per seed values run from 2.2 to 28.7. The values are not scattered evenly across that interval. 5 seeds cluster near 27 times and 3 sit near 3 times, which is why the mean falls below the median.

Key Insight

The direction of the Week 3 result is solid: the RR layer generates better than the baseline on every one of 8 seeds. The magnitude is not. It ranges from 2.2 to 28.7 times and splits into two groups rather than varying smoothly, so the claim for the final report should be a range together with the observation that the size of the gain depends on whether the RRAE trained cleanly.

4.5 Two regimes across seeds

Sorting the seeds by the reconstruction error the RRAE finally reached splits them cleanly into two groups. On 5 seeds the RRAE settles near 0.011 and beats the baseline on generation by about 27 times, while also beating it on interpolation. On the remaining 3 seeds it settles near 0.063, roughly 5 times worse, and its generative advantage falls to about 3 times while its interpolation error becomes worse than the baseline. The two groups are visible as two clusters in Figure 4. Nothing distinguishes the seeds except initialization, so the most plausible reading is that optimizing through a hard rank 2 projection sometimes lands in a poorer optimum, and that when it does, the quality of the latent space degrades with it. Report 3 used three seeds and, judging by its reported reconstruction figures, drew all three from the well converged group.

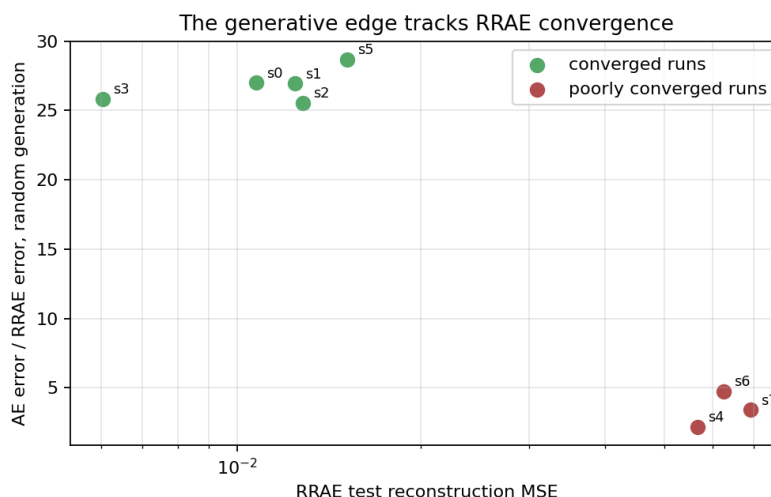


Figure 4 Generative advantage against final RRAE reconstruction error, one point per seed. The seeds on which the RR layer trained cleanly are also the seeds on which its generative advantage is largest.

4.6 Why the RR layer generates better

The last column of Table 2 is the mechanism. The RRAE puts 100 percent of its latent variance in two directions, by construction. The baseline puts 52 percent and the batch normalized model only 24 percent. When we sample from a Gaussian fitted to the latent codes, the RRAE can only produce codes that lie in the subspace its decoder was trained on, whereas the other two spread mass over directions that carry little training signal and decode into images that are not blobs. The two degrees of freedom of the data and the rank of the layer match exactly here, which is the most favourable case the method can be given.

5 Discussion

The Week 3 direction holds, the Week 3 number does not. The replication was written from scratch, with its own data generator, its own model code and its own metric definitions, and it still reproduces the ordering Report 3 found: the RR layer is the only one of the three that turns a random draw from its latent space into something that looks like a valid sample. The size of the advantage, however, moved: a mean of 18.0 times over 8 seeds against the 7.9 times of Report 3, with individual seeds anywhere between 2.2 and 28.7 times. The lesson is about reporting rather than about the method. A mean ratio computed over three seeds looked like a property of the model; seen over 8 seeds it is clearly a distribution, and the honest statement of the result names the range.

The gain comes in two regimes. The clearest new finding of this report is not the size of the generative gain but its bimodality. 3 of 8 runs of the RRAE ended in a visibly worse optimum, and in exactly those runs the generative advantage shrank from about 27 times to about 3 times and the interpolation advantage disappeared. The RR layer therefore has a training stability problem on this setup that the three seed sweep of Report 3 could not have detected. This is worth raising with the mentor, since it concerns how the layer is optimized rather than how it is evaluated.

The reconstruction penalty is real and is the price of the method. The RRAE reconstructs a median factor of 21 worse than the baseline at 40 epochs, consistent in sign on every seed. This is

not a defect to be tuned away. Rank 2 is a hard constraint: the layer projects the entire batch onto a two dimensional subspace before decoding, so the autoencoder cannot store per-sample detail that lies outside that subspace. The baseline is free to spread information across all 200 latent units and therefore reconstructs better, and for exactly the same reason its latent space contains large regions that decode to nothing meaningful.

Why batch normalization keeps losing. Batch normalization has now lost in four consecutive report cycles. Here it is 2.8 times worse than the plain baseline on random generation and worse on interpolation as well. Batch normalization equalizes the scale of the latent coordinates without changing which directions carry information, and on this dataset that is the wrong operation twice over. The task has two underlying degrees of freedom, the two coordinates of the centre, so the useful structure is a question of directions, not of scale. Rescaling every direction to unit variance amplifies the low variance directions, which here carry mostly noise, and this shows up directly as the lowest top 2 energy of the three models. The RR layer does the opposite: it discards those directions entirely.

Relation to the underlying claim. The paper behind this layer motivates rank reduction as a way to obtain a latent space with a controlled dimension in which samples lie on a compact manifold [3]. Our results are consistent with that: the advantage appears exclusively on the metrics that probe the latent space, and the top 2 energy column explains it without appeal to anything else. What our results add is a caution. On a two degree of freedom problem, where rank 2 is exactly the right rank, the effect is large but its magnitude varies by an order of magnitude across seeds. Any claim of a specific speedup or improvement factor from this family of experiments should be quoted as an interval.

6 Code and Repository Status

Two repositories are relevant to this project. The reference implementation of the RR layer is our mentor's [4], at https://github.com/JadM133/RR_layer. The team fork is [7], at <https://github.com/Yousef10p/RR-Layer>, where the live branch is new-experiment; the main branch still holds Week 1 material only.

Three items need attention before the final report.

- 1. Week 3 and Week 4 code is not yet pushed.** Neither branch contains the autoencoder and generative evaluation code. The script accompanying this report, `run_w4.py`, is self contained and should be pushed to new-experiment together with the Week 3 material.
- 2. The eight seed results already on new-experiment used the old SVD implementation.** Those numbers predate the switch to the native `torch.linalg.svd` wrapper made in Week 2 and should not be carried into the final report. The runs in this report use the current implementation.
- 3. Upstream findings for the mentor.** Report 3 collected three: debug prints left live inside the SVD backward pass, the redundant `finalize_basis_from_data` addition in our fork, and a spurious rank warning raised at single sample inference in evaluation mode. None of them affect the numbers reported here, and the second is a negative result about our own fork rather than about the reference code.

7 Limitations and Future Work

One dataset, one rank, one latent width. Every result in this project comes from Gaussian blobs on a 64×64 grid with a 200 dimensional latent and an RR layer of rank 2. The problem has exactly two degrees of freedom, so rank 2 is the ideal setting and the results are close to a best case for the method. The obvious next step is a rank sweep, $k \in \{1, 2, 4, 8, 16\}$, to see how quickly the generative advantage decays once the rank exceeds the intrinsic dimension of the data, followed by a dataset whose intrinsic dimension is not known in advance.

Training budget. The sweep uses 40 epochs so that 24 runs fit in the available compute. Report 3 found that the reconstruction gap between RRAE and the baseline shrinks substantially at 200 epochs, so the reconstruction column of Table 2 should be read as an upper bound on the penalty rather than its converged value. Repeating this sweep at the longer budget is the single most useful missing measurement, and it is a matter of compute time rather than of new code.

Seed count and statistical treatment. 8 seeds are enough to show that the generation advantage is consistent in sign and wide in magnitude, but not enough to characterize the shape of that distribution. Reporting a paired test across seeds, and enough seeds to support it, would let the final report state a confidence interval rather than a range.

Training stability of the RR layer. Section 4.5 shows that a fraction of RRAE runs end in a poorer optimum, and that the latent space quality degrades with them. We did not investigate why. Candidate next steps, in increasing order of effort: warm up the model for a few epochs before switching the RR layer on, average the basis over a short window rather than recomputing it per batch, and check whether the poor runs are recoverable with a lower learning rate. Any of these would tell us whether the effect is an optimization artefact or a property of the layer.

Metric design. Both latent metrics compare shapes after per-image standardization, which deliberately ignores intensity. A model that produces a perfectly shaped blob at the wrong contrast is not penalized. This choice was made so that the three models are compared on structure alone, but a complementary metric that scores position accuracy of generated samples against the training distribution of centres would round out the picture.

8 Conclusion

Four weeks of experiments give a consistent picture. The RR layer is a better choice than batch normalization everywhere we have looked, on the localization task of Reports 1 and 2 and on all three metrics of the autoencoder setting. Against a plain baseline it is a trade rather than an improvement: it costs a median factor of 21 in reconstruction and buys a latent space whose random samples decode into valid images, 18.0 times better than the baseline on average over 8 seeds. The most important correction this report makes to the previous one is that the advantage is not the single clean factor Report 3 quoted; the per seed range is 2.2 to 28.7 times, and the mean is a summary of a wide distribution rather than a property of the method. The takeaway for the final report is that the RR layer should be presented as a way to buy latent structure at a known cost in reconstruction accuracy, with the size of the gain stated as an interval.

References

- [1] Sergey Ioffe and Christian Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015, pp. 448–456.
- [2] Carl Eckart and Gale Young. "The approximation of one matrix by another of lower rank". In: *Psychometrika* 1.3 (1936), pp. 211–218.
- [3] Jad Mounayer et al. "Rank Reduction Autoencoders". In: *arXiv preprint arXiv:2405.13980* (2024). URL: <https://arxiv.org/abs/2405.13980>.
- [4] Jad Mounayer. *RR_layer: reference implementation of the Rank Reduction layer*. https://github.com/JadM133/RR_layer. Accessed: 2026-08-15.
- [5] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. 2015.
- [6] Adam Paszke, Sam Gross, Francisco Massa, et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019.
- [7] Yousef Alogiley, Abdulmohsen Mohammedsaleh, and Majed Alsulami. *RR-Layer: KAUST Academy team fork, branch new-experiment*. <https://github.com/Yousef10p/RR-Layer/tree/new-experiment>. Accessed: 2026-08-15.